

Prompt 3 — Cablear el catálogo semántico

Va **antes** de construir vistas. Definir los nombres después de tener las pantallas es exactamente cómo se llega al despelote que este proyecto quiere evitar.

Repo de especificación: `github.com/leanchoi/leanchoi`, rama `claude/esquel-data-ecosystem-integration-qbxuv2`.

Leé `AGENTS.md` y `docs/06-productivizacion.md` COMPLETO. Es el documento central del proyecto y el que más se malinterpreta si se lee por arriba.

OBJETIVO

Que `specs/catalogo/indicadores.yaml` sea la fuente única de verdad de todo el ecosistema, con imposibilidad ESTRUCTURAL de que el tablero, el ETL y la documentación diverjan.

TAREAS

1. Integrar el generador al proyecto real.

`specs/scripts/gen_catalogo.py` ya funciona y está probado. Ajustá las rutas de salida a las del proyecto:

```
web/src/generated/indicadores.ts
etl/generated/indicadores.py
docs/generated/ficha-metodologica.md
```

Agregá ambos directorios `generated/` al control de versiones (se `commitean`: es lo que permite que `--check` detecte divergencias).

2. Verificación en CI / pre-commit.

``python3 specs/scripts/gen_catalogo.py --check`` debe fallar el build si lo generado difiere de lo `commiteado`. Sin esto el catálogo es documentación, no contrato.

3. Cablear la validación al ETL.

En `etl/emit.py`, antes de escribir CADA tabla oro nueva:

```
from etl.generated.indicadores import validar_columnas
faltantes = validar_columnas(nombre_tabla, list(df.columns))
if faltantes: -> abortar SOLO esa tabla, marcarla no disponible, seguir el build
```

Una columna que no está en el catálogo es un número sin definición, sin dueño y sin decisión asociada (I11). Las 24 tablas preexistentes quedan EXENTAS: no las toques (I2).

4. Cablear el semáforo de confianza al frontend.

Usá `esPublicable()` y `esPreliminar()` del TS generado:

- grado C -> se muestra siempre como banda, nunca como punto
- grado D o cobertura bajo el mínimo -> no se muestra
- cobertura < 80% -> la serie se marca preliminar visualmente
- sin medición -> estado "sin señal", NUNCA verde (I13)

Respetá tokens.css y el sistema de diseño existente. No introduzcas una librería de UI.

5. Extender meta.json.

Por dataset: disponible, filas, actualizado, cobertura_7d, schema_version, motivo, ultimo_ok. Contrato del frontend, tres reglas:

- a. Consultar meta.json ANTES de cargar un .arrow. Si no está disponible, la sección muestra su estado y la fecha del último dato bueno. Nunca pantalla en blanco (I4).
- b. cobertura_7d < 0,80 -> series marcadas preliminares.
- c. schema_version distinta de la esperada -> sección deshabilitada con aviso de "actualizá la página". Un desajuste de despliegue debe ser un mensaje, no un crash.

6. Motor de insights.

etl/modelos/insights.py evalúa specs/catalogo/insights.yaml y emite x_fact_insights.

- Si falta un indicador de `requiere`, la regla SE OMITE y se registra por qué. No falla, y sobre todo no inventa.
- El frontend RENDERIZA tarjetas. Cero lógica de diagnóstico en componentes (I14).
- Un test sintético por regla, con resultado conocido.

CRITERIOS DE ACEPTACIÓN

1. gen_catalogo.py --check en verde y corriendo en CI.
2. Agregar a mano una columna fuera del catálogo aborta esa tabla y solo esa; el build termina en verde y emite el resto.
3. Un indicador grado C se renderiza como banda; uno D no se renderiza.
4. Con meta.json marcando un dataset no disponible, la sección informa y el resto del tablero funciona normal.
5. Las 24 tablas preexistentes salen byte-idénticas (sha256sum antes y después).
6. La ficha metodológica generada está commiteada y sincronizada.